

Glade User Interface Designer Reference Manual

Glade User Interface Designer Reference Manual

for gladeui version 2.0;.

This document attempts to describe how to use the Glade UI Designer core library to integrate Glade into your application and how to integrate your custom GTK+ derived widgets into the Glade UI Designer. The latest version of this documentation can be found on-line at

<https://developer.gnome.org/gladeui/stable>

Table of Contents

1 Adding your custom widgets.....	4
1.1 Writing catalogs.....	4
1.1.1 Introduction.....	4
1.1.2 Top level catalog properties and tags.....	5
1.1.3 Validating and installing.....	6
1.2 Widget Classes.....	7
1.2.1 Forward.....	7
1.2.2 Widget Class Properties.....	8
1.2.3 Adapter Methods.....	10
1.2.4 Action Definitions.....	13
1.2.5 Packing Defaults.....	13
1.2.6 Signals.....	14
1.2.7 Icons.....	14
1.3 Property Definitions.....	14
1.3.1 Property Definition Configuration.....	14
1.3.2 Properties of the 'property' tag.....	15
1.3.3 Child tags of the 'property' tag.....	17
1.4 Python Gtk Widgets Support.....	17
1.4.1 Introduction.....	17
1.4.2 pythonplugin.py.....	18
1.5 JavaScript Gtk widgets support.....	18
1.5.1 Introduction.....	19
1.5.2 gjsplugin.js.....	19
2 Glade Core.....	21
3 Dockable Glade UI Widgets.....	22
3.1 Glade Editable.....	22
3.2 Glade Editor — A Widget to edit a GladeWidget.....	22
3.3 Glade Editor Property — A generic widget to edit a GladeProperty.....	22
3.4 GladePalette— A widget to select a GladeWidgetAdaptor for addition.....	22
3.5 GladeInspector— A widget for inspecting objects in a GladeProject.....	22
3.6 GladeSignalEditor— An interface to edit signals for a GladeWidget.....	22
3.7 GladePreviewer.....	22
3.8 GladeDesignView— A widget to embed the workspace.....	22
3.9 GladePreview— The glade preview launch/kill interface.....	22
3.10 GladeAdaptorChooser.....	22
3.11 GladePropertyLabel.....	22
3.12 GladePropertyShell.....	23
4 Miscellaneous utilities.....	24
4.1 GladeCellRendererIcon.....	24
4.1.1 Functions.....	24
4.1.2 Types and Values.....	24
4.1.3 Description.....	24

4.1.4 Functions.....	24
4.1.5 Types and Values.....	26
4.2 GladeObjectStub.....	26
4.2.1 Types and Values.....	26
4.2.2 Description.....	26
4.2.3 Functions.....	26
4.2.4 Types and Values.....	26
4.3 GladeSignalModel.....	26
4.4 Glade Utils.....	26
5 Index.....	27
6 Index of deprecated symbols.....	28

1 Adding your custom widgets

1.1 Writing catalogs

How to write and install a catalog.

1.1.1 Introduction

You can provide support for your custom widgets in a few ways, you can make a package and install it to the system directories, load additional catalogs in user directories, project directories for example, and you can optionally provide code support and/or icons, normally you need to at least have the object type in a library somewhere, but you can work around this using the 'parent' property described in the next section. If you don't provide icons for the inspector and palette Glade will simply print a warning and use a default icon. The catalog file is written in an XML format and a DTD for the format can be found in the plugins/ directory of the Glade tarball.

In most cases gtk+ derived widgets can be added with little effort and it is enough to simply specify the widget's type; glade will introspect its properties and signals - but due to the organic nature of a widget toolkit there are always exceptions. In this document we'll try to provide some basic examples and describe a wealth of options that can be used to enhance UI editing and workaround exceptions.

Keep in mind you need to take extra steps to ensure GtkBuilder can pick up your new object types at runtime. Usually all you need is to link your executable with your widget library assuming it has properly named `get_type()` functions:

```
GtkWindow -> gtk_window_get_type
GtkHBox → gtk_hbox_get_type
GtkUIManager -> gtk_ui_manager_get_type
```

If not, you can always register your widgets with the type system with `g_type_ensure()`

The catalog file starts by specifying the name of the catalog and the plugin library to use, the following examples assume you have a namespace "Foo" and are integrating an object "Frobnicator":

```
<glade-catalog name="foo" library="foo" depends="gtk+">
  <init-function>my_catalog_init</init-function>
```

```

<glade-widget-classes>
  <glade-widget-class name="FooFrobnicator"
    generic-name="frobnicator"
    title="Frobnicator"/>

  ... widget classes go here
</glade-widget-classes>

<glade-widget-group name="foo" title="Foo">
  <glade-widget-class-ref name="FooFrobnicator"/>
  ... widget class references go here
</glade-widget-group>

... widget groups go here
</glade-catalog>

```

1.1.2 Top level catalog properties and tags

When defining the catalog, the 'name' and 'library' are both mandatory attributes of the 'glade-catalog' tag; optionally you can also use 'icon-prefix', 'depends' and 'domain'.

name

A string identifier for the catalog in question, it will be used to identify your catalog so that the glade file can explicitly require it and to manage inter catalog dependencies.

version

A 'major.minor' formed version describing the current version of underlying widget kit; example: version="1.0". This is needed for version checking to work. Please note that all versioning related support is completely optional.

targetable

A comma separated list of 'major.minor' formed versions describing sensible previous targetable versions of the underlying toolkit not including the current version; example: targetable="0.6,0.8".

icon-prefix

Used to form icon names for widgets. This property defaults to the value of the 'name' attribute.

library

Used to load the types and introspect properties, unless you are faking your widget classes (which will be described later on), glade will need to load this library, it can either be the name of the library containing the widgets or the plugin library which is assumed to implicitly link to your widget library. The library will be loaded either by a user specified path, the system plugin directory: `$prefix/lib/glade-3/modules/`, or from the default system library paths in the afore mentioned order of precedence.

depends

Used for inheritance of support code to work properly (i.e. if your object derives from an object in gtk+, you'll want the default support code in the gladegtk plugin to be enabled for your widget too). This property's value is the `'name'` property of another installed glade plugin; usually you'll want to declare: `'depends="gtk+"'` for your plugin.

domain

The domain in which to search for translatable strings from the catalog file; please note that all strings from the catalog that will appear in the UI are translated using this domain. If the `'domain'` is not specified, the library property will be used in it's stead.

book

Used to specify a namespace to search devhelp docs library with (specifically, it is the `$ (DOC_MODULE)` that you specified in your `gtk-doc Makefile.am`).

init-function

Used to retrieve an optional global entry point to your plugin; if you need to initialize any backends or what not, this is a good place. Your catalog's `init-function` will be called before any widget classes are instantiated.

1.1.3 Validating and installing

The DTD that is shipped with Glade can be used to validate your catalog file. Note that properties must be entered in the same order as they are specified in the DTD for the validation to pass.

To validate a file, do this:

```
xmllint --dtdvalid glade-catalog.dtd --noout my-catalog.xml
```

To install a widget plugin, the catalog XML file should be copied into the catalog directory, which can be retrieved as:

```
pkg-config --variable=catalogdir gladeui-1.0
```

The plugin library should be installed into the modules directory:

```
pkg-config --variable=moduledir gladeui-1.0
```

Widget icons if provided (recommended) need to be installed into the icon theme, this is described in the next chapter.

You can also load your catalog from a user directory by specifying additional load path(s) in the environment, for instance:

```
GLADE_CATALOG_SEARCH_PATH=~/.mycatalogs:~/work/foo/glade
```

Same goes for optional plugin libraries, for instance:

```
GLADE_MODULE_SEARCH_PATH=~/.work/foo/src
```

Currently loading icons without installing them is unsupported.

1.2 Widget Classes

Adding support for custom widgets.

1.2.1 Forward

Widget support in Glade is provided by the #GladeWidgetAdaptor object, each 'glade-widget-class' definition in the catalog will result in the creation of an adaptor, which inherits the functionality of the adaptor created for its parent, or the closest adaptor

available in the ancestry. Adaptor methods can be overridden by functions in the plugin by specifying them in the catalog. There is also a wealth of options you can use in the ``glade-widget-class'` without writing code.

A typical basic example of a widget class definition is as follows:

```
<glade-widget-class name="FooFrobnicator"
                    generic-name="frobnicator"
                    title="Frobnicator"/>
```

Here is the complete example of the form of the ``glade-widget-class'` definition:

```
<glade-widget-class name="FooFrobnicator"
                    generic-name="frobnicator"
                    title="Frobnicator"/>
<post-create-function>foo_frobnicator_plugin_post_create
</post-create-function>

... widget class support functions go here

<properties>
  <property id="froblicated" default="False"/>

  ... property definitions go here
</properties>

<signals>
  <signal id="frobnicate" since="1.0"/>

  ... signal definitions go here
</signals>

<packing-properties>
  ... child packing property definitions go here
</packing-properties>

<actions>
  <action id="launch_editor" name="Edit Frob"
          stock="gtk-edit"/>

  ... context menu action definitions go here
</actions>

<packing-actions>
  ... context menu action definitions for child
                                objects go here
</packing-actions>

<packing-defaults>
  <parent-class name="GtkVBox">
    <child-property id="expand" default="false"/>
```

```
</parent-class>

... packing default definitions go here
</packing-defaults>
</glade-widget-class>
```

1.2.2 Widget Class Properties

The following are all properties of the ``glade-widget-class'` tag:

name

Mandatory. The class name of the widget; this will essentially be used to instantiate the actual class by deriving, for example, if name is 'GtkLabel' then 'gtk_label_get_type' is derived and Glade will search for 'gtk_label_get_type' in the support library. It takes precedence over the 'get-type-function' property.

since

A 'major.minor' formed version describing the version of the owning catalog in which this object class was introduced; example: `since="1.0"`.

deprecated

A boolean property depicting that this widget is currently deprecated.

builder-unsupported

A boolean property depicting that this widget is not supported by #GtkBuilder.

get-type-function verify

Used to explicitly specify the name of the function used to get the type of the widget. It is optional, but if the 'name' property is present, it takes precedence over this function and such name will be used to guess the name of the function, a process that could lead to unexpected results.

generic-name

Used to generate a default name for instances of the widget in the UI editor. It is also used in conjunction with the 'icon-prefix' to form an icon name for the widget.

icon-name

Used to explicitly set an icon name for this widget. These icon names are looked up in the current icon theme to retrieve an icon for the widget.

title

Used to display the name of the class in the palette and widget tree and will be translated before use in the interface.

parent

Use this to specify the name of the parent your widget derives from, you can use this option instead of loading a real widget type (this allows you to fake derive and still add properties and run your catalog independently of any plugins or libraries).

oplevel

Used to know whether this widget class is top level or not in Glade context. This property will be inherited from the adaptor's parent.

fixed

Used to mark a #GladeWidgetAdaptor for free form placement capacities to be handled with a #GladeFixed. This property will be inherited from the adaptor's parent.

use-placeholders

Used to inform the core about adaptors that implement #GtkContainer editing, and whether or not they use #GladePlaceholder. This property will be inherited from the adaptors parent.

default-width

The default width to load a #GtkWindow derived widget in the #GladeDesignView. This property will be inherited from the adaptor's parent.

default-height

The default height to load a #GtkWindow derived widget in the #GladeDesignView. This propertyList Heading will be inherited from the adaptor's parent.

1.2.3 Adapter Methods

The following are all child tags of the ``glade-widget-class'` tag:

constructor-function

Used to override the actual `#GObject` constructor of the said `#GladeWidgetAdaptor`

post-create-function

A `#GladePostCreateFunc` support function that gets called whenever a widget of the said class is instantiated. These are the reasons your `#GladePostCreateFunc` can be called:

- `GLADE_CREATE_USER`: Was created at the user's request (this is a good time to set any properties or add children to the project; like `GtkFrame`'s label for example).
- `GLADE_CREATE_COPY`: Was created as a result of the copy/paste mechanism, at this point you can count on glade to follow up with properties and children on its own.
- `GLADE_CREATE_LOAD`: Was created during the load process.
- `GLADE_CREATE_REBUILD`: Was created as a replacement for another project object; this only happens when the user is changing a property that is marked by the type system as `G_PARAM_SPEC_CONSTRUCT_ONLY`.
- `GLADE_CREATE_REASONS`: Never used.

deep-post-create-function

Same as ``post-create-function'` except in that you must always chain up to the super class definition as a rule.

get-property-function

A `#GladeGetPropertyFunc` to get values of properties on an instance in the runtime.

set-property-function

A `#GladeSetPropertyFunc` to set values of properties on an instance in the runtime.

verify-function

A #GladeVerifyPropertyFunc to set values of properties on an instance in the runtime. This delegate function is always called whenever setting any properties with the exception of load time, and copy/paste time (basically the two places where we recreate a hierarchy that we already know "works"). It is basically an optional backend provided boundary checker for properties. Returns whether or not its OK to set @value on @object

add-child-function

A #GladeAddChildFunc to add children to instances in the runtime.

remove-child-function

A #GladeRemoveChildFunc to remove children from instances in the runtime.

replace-child-function

A #GladeReplaceChildFunc to replace children inside instances in the runtime.

get-children-function

A #GladeGetChildrenFunc to get a list of children inside instances in the runtime.

child-get-property-function

A #GladeChildGetPropertyFunc to get packing properties from instances in the runtime.

child-set-property-function

A #GladeChildSetPropertyFunc to set packing properties on instances in the runtime.

child-verify-function

A #GladeChildVerifyPropertyFunc to verify packing properties on instances in the runtime.

get-internal-child-function

A #GladeGetInternalFunc to retrieve an internal child of a composite object in the runtime.

action-activate-function

A #GladeActionActivateFunc to run a routine for a plugin defined #GladeWidgetAction.

child-action-activate-function

A #GladeChildActionActivateFunc to run a routine for a plugin defined packing #GladeWidgetAction.

read-widget-function

A #GladeReadWidgetFunc to load a widget from the xml.

write-widget-function

A #GladeWriteWidgetFunc to write a widget to the xml.

read-child-function

A #GladeReadWidgetFunc to read a child from the xml.

write-child-function

A #GladeWriteWidgetFunc to write a child to the xml.

create-editor-property-function

A #GladeCreateEPropFunc to create widgets to be used in Glade's property editor.

string-from-value-function

A #GladeStringFromValueFunc to create a string from a #GValue, these strings must be unique and comparable for undo/redo to work properly.

1.2.4 Action Definitions

Actions are added under the `actions' and `packing-actions' tags, actions can also be nested so as to allow grouping in context menus, example:

```
<action id="manage_frobs" name="Manage Frobs">
  <action id="add_frob" name="Add" stock="gtk-add"/>
  <action id="remove_frob" name="Remove" stock="gtk-remove"/>
</action>
```

Here are the meanings of the `action' tag's properties:

id

The string identifier for this action, the `action_path` argument to your `#GladeActionActivateFunc` will be a combination of this id and any parents, for the above example: `"manage_frobs/add_frob"`.

name

A string to be displayed in the UI for this action.

stock

An optional Gtk stock icon to represent this action.

important

A boolean option to add emphasis to this action; currently actions marked as `important' are added to the toolbar.

1.2.5 Packing Defaults

Default values can be specified for packing properties on your widget when added to a said type of parent, as the example shows above; use the `parent-class' to specify what parent (or derivative of that parent) you are targeting and fill in any `child-property' tags you need for that parent using `id' property to specify the property name and the `default' property for the value.

1.2.6 Signals

The signal elements are currently only needed for versioning support, use the `signal' child tag as described above and set signal `id' to be supported since version `since'. Signals are initialized to be supported since the introducing `#GladeWidgetAdaptor` was supported.

1.2.7 Icons

The Glade palette and other components use icons to represent the various widget classes. It is a good idea to provide icons for your widget classes, as this enhances the user interface.

Glade uses the GTK+ `GtkIconTheme` facility to provide icons for widget classes. Any icon available in the current icon theme can be used as an icon for a widget class.

By default, an icon name of the format "widget-CATALOG_NAME-GENERIC_NAME" is assigned to every widget class. CATALOG_NAME is the value of catalog name attribute, and GENERIC_NAME is the value of an widget class's generic name attribute.

To explicitly set an icon name for a widget class, the "icon-name" attribute of the "glade-widget-class" element can be specified. This will override the default icon name assigned to the widget class.

Icon files can be installed under any system icon theme folder.

1.3 Property Definitions

How to augment or define a #GladePropertyDef.

1.3.1 Property Definition Configuration

Properties are introspected at load time and loaded into #GladePropertyDef structures. The behaviour of properties can be modified by the catalog and fake properties can be added for editing purposes. Here is an example of the xml form:

```
<property id="frobnicate-mode" default="FOO_FROBNICATE_RED">
  ... spec, tooltip etc
  <displayable-values>
    <value id="FOO_FROBNICATE_RED" name="Red"/>
    <value id="FOO_FROBNICATE_BLUE" name="Blue"/>
    ... values here
  </displayable-values>
</property>
```

1.3.2 Properties of the 'property' tag

id

This is mandatory and specifies the property that we are modifying (or adding)

name

The name to be used in the interface. (if name is not specified; it defaults to the

nickname of the #GParamSpec)

since

A 'major.minor' formed version describing the version of the owning catalog in which this property was introduced; example: since="1.0". Properties are initialized to be supported since the introducing #GladeWidgetAdaptor was supported.

disabled

Remove this property from this widget class and derived classes

default

A default value to be used for this property

translatable

For text properties, whether the property value is translatable in glade interfaces (this will enable the i18n dialog on text properties). Defaults to False.

common

If set to "True", the property will end up on the common tab even if its not a property of GtkWidgetClass.

optional

Whether this property is an optional property, this will make the property insensitive and add a check box to enable it (like width/height-request for example).

optional-default

If this is in fact an optional property; whether it is enabled by default.

query

If query is set; the property will be queried from the user in a dialog when adding the owning widget class instance to the project.

save

Whether to save this property to the glade file (default "True")

visible

Whether to show the property in the editor and reset dialog (default "True")

custom-layout

This is used to avoid loading this property in the editor when implementing a custom #GladeEditable that embeds the base #GladeEditorTable implementation, custom-layout properties will still show up in the reset dialog (default "False")

ignore

Whether to set the property on the object instance (via g_object_set_property or plugin override functions) when it changes in the editor (the value in the editor is the value saved).

themed-icon

Depicts a string property that is used for an icon from the theme. These will the appropriate editor.

weight

A numerical value to specify this properties position in the property editor.

transfer-on-paste

Used for packing properties; depicts packing properties that should follow the widget when pasted to a new container that supports the same properties.

save-always

Specifies that the property should be saved regardless of its value (properties at their default values are normally not saved).

1.3.3 Child tags of the 'property' tag

spec

Specifies a function to be used to return a #GParamSpec for this property; this is used to add virtual properties to an object (like the "size" property on #GtkBox).

tooltip

The tooltip to be displayed in the property editor for this property. The tooltip defaults

to the blurb of the associated #GParamSpec.

visible-lines

An integer value to specify how many lines will be shown for text properties in the editor (this doesn't really work because of the complexity of calculating size of rendered text; instead, just set this to 2 if you want the text property to be edited in a textview with a scrolled window as opposed to a simple text entry).

displayable-values

Allows you to specify user-friendly strings for enum and flag values as shown in the example above, use the ``id'` property in the value tag to depict the real value name and the ``name'` property for the human-readable one.

1.4 Python Gtk Widgets Support

Add python support to your catalog — How to write and install a catalog for a python widget library.

1.4.1 Introduction

Glade supports loading widgets coded in python by linking and running the python interpreter from the gladepython catalog plugin.

So in order for glade to include your python gtk widgets you will have to:

- a) specify gladepython support code as your plugin library.
- b) set `glade_python_init` as you init function.
- c) make sure your catalog name is the same as your python import library since `glade_python_init()` will use this name to import your widgets into the interpreter. `pythonplugin.xml`

```
<glade-catalog name="pythonplugin" library="gladepython"
domain="glade-3" depends="gtk+">
  <init-function>glade_python_init</init-function>

  <glade-widget-classes>
    <glade-widget-class title="MyPythonBox" name="MyPythonBox"
```

```

generic-name="mypythonbox"/>
</glade-widget-classes>

<glade-widget-group name="python" title="Python">
  <glade-widget-class-ref name="MyPythonBox"/>
</glade-widget-group>
</glade-catalog>

```

Glade's python interpreter will look up for your widgets in the same places it looks for regular catalogs plugins, that is `$GLADE_ENV_MODULE_PATH` environment variable and ``pkg-config --variable=moduledir gladeui-2.0`` So the easiest thing would be to make a symlink in one of those directory, just do not forget that the name should be the one specified in your catalog name.

1.4.2 pythonplugin.py

```

from gi.repository import GLib, Gtk, GObject

class MyPythonBox(Gtk.Box):
    __gtype_name__ = 'MyPythonBox'

    foo = GObject.Property(type=int, nick='An integer')
    bar = GObject.Property(type=str, nick='A String')

    def _update(self, obj, pspec):
        self.label.set_text ('Python Properties\nInteger = ' +
str(self.foo) + '\nString = \'' + self.bar + '\'')

    def __init__ (self):
        Gtk.Box.__init__ (self)
        self.label = Gtk.Label (visible = True)
        self.add (self.label)
        self.connect('notify::foo', self._update)
        self.connect('notify::bar', self._update)
        self._update(None, None)

```

1.5 JavaScript Gtk widgets support

Add GJS/JavaScript support to your catalog — How to write and install a catalog for a JavaScript widget library

1.5.1 Introduction

Glade supports loading widgets programmed in JavaScript by linking and running GJS from the glade-gjs catalog plugin.

So in order for glade to support your JavaScript widgets you will have to:

- a) specify glade-gjs support code as your plugin library.
- b) set `glade_gjs_init` as you init function.
- c) make sure your catalog name is the same as your JavaScript import library since `glade_gjs_init()` will use this name to import your widgets into the interpreter.

```
<glade-catalog name="gjsplugin" library="gladegjs" domain="glade-3"
depends="gtk+">
  <init-function>glade_gjs_init</init-function>

  <glade-widget-classes>
    <glade-widget-class title="MyJSGrid" name="MyJSGrid" generic-
name="mygrid"/>
  </glade-widget-classes>

  <glade-widget-group name="gjs" title="Gjs">
    <glade-widget-class-ref name="MyJSGrid"/>
  </glade-widget-group>
</glade-catalog>
```

GJS will look up for your widgets in the same places it looks for regular catalogs plugins, that is `$GLADE_ENV_MODULE_PATH` environment variable and ``pkg-config --variable=moduledir gladeui-2.0`` So the easiest thing would be to make a symlink in one of those directory, just do not forget that the name should be the one specified in your catalog name.

1.5.2 gjsplugin.js

```
#!/usr/bin/gjs

const GObject = imports.gi.GObject;
const Gtk      = imports.gi.Gtk;

var MyJSGrid = GObject.registerClass({
  GTypeName: 'MyJSGrid',
  Properties: {
    'string-property': GObject.ParamSpec.string('string-property',
```

```

'String Prop',
  'Longer description', GObject.ParamFlags.READWRITE |
GObject.ParamFlags.CONSTRUCT,
  'Foobar'),
  'int-property': GObject.ParamSpec.int('int-property', 'Integer
Prop',
  'Longer description',
  GObject.ParamFlags.READWRITE |
GObject.ParamFlags.CONSTRUCT,
  0, 10, 5)
},
Signals: {'mysignal': {param_types: [GObject.TYPE_INT]}},
}, class MyJSGrid extends Gtk.Grid {
  _init(props) {
    super._init(props);
    this.label = new Gtk.Label ({ visible: true });
    this.add (this.label);
    this.connect('notify::string-property',
this._update.bind(this));
    this.connect('notify::int-property',
this._update.bind(this));
    this._update();
  }
  _update (obj, pspec) {
    this.label.set_text ('JS Properties\nInteger = ' +
this.int_property + '\nString = \'' + this.string_property + '\');
  }
});

```

2 Glade Core

3 Dockable Glade UI Widgets

3.1 Glade Editable

3.2 Glade Editor — A Widget to edit a GladeWidget.

3.3 Glade Editor Property — A generic widget to edit a GladeProperty.

3.4 GladePalette— A widget to select a GladeWidgetAdaptor for addition.

3.5 GladeInspector— A widget for inspecting objects in a GladeProject.

3.6 GladeSignalEditor— An interface to edit signals for a GladeWidget.

3.7 GladePreviewer

3.8 GladeDesignView— A widget to embed the workspace.

3.9 GladePreview— The glade preview launch/kill interface.

3.10 GladeAdaptorChooser

3.11 GladePropertyLabel

3.12 [GladePropertyShell](#)

4 Miscellaneous utilities

4.1 GladeCellRendererIcon

4.1.1 Functions

gboolean [glade_cell_renderer_icon_get_activatable \(\)](#)

gboolean [glade_cell_renderer_icon_get_active \(\)](#)

[GtkCellRenderer](#) * [glade_cell_renderer_icon_new \(\)](#)

void [glade_cell_renderer_icon_set_activatable \(\)](#)

void [glade_cell_renderer_icon_set_active \(\)](#)

4.1.2 Types and Values

[GladeCellRendererIcon](#)

4.1.3 Description

4.1.4 Functions

glade_cell_renderer_icon_get_activatable ()

gboolean

```
glade_cell_renderer_icon_get_activatable  
    (GladeCellRendererIcon *icon);
```

glade_cell_renderer_icon_get_active ()

```
gboolean  
glade_cell_renderer_icon_get_active (GladeCellRendererIcon *icon);
```

glade_cell_renderer_icon_new ()

```
GtkCellRenderer *  
glade_cell_renderer_icon_new (void);
```

glade_cell_renderer_icon_set_activatable ()

```
void  
glade_cell_renderer_icon_set_activatable  
    (GladeCellRendererIcon *icon,  
     gboolean setting);
```

glade_cell_renderer_icon_set_active ()

```
void  
glade_cell_renderer_icon_set_active (GladeCellRendererIcon *icon,  
     gboolean setting);
```

4.1.5 Types and Values

GladeCellRendererIcon

```
typedef struct _GladeCellRendererIcon GladeCellRendererIcon;
```

4.2 GladeObjectStub

4.2.1 Types and Values

[GladeObjectStub](#)

4.2.2 Description

4.2.3 Functions

4.2.4 Types and Values

GladeObjectStub

```
typedef struct _GladeObjectStub GladeObjectStub;
```

4.3 GladeSignalModel

4.4 Glade Utils

5 Index

6 Index of deprecated symbols